

Git Workflow — Reference Guide (Phase 2)

How git is actually used day-to-day at an org, mapped to commands you already know plus the ones you don't yet.

1. The mental model

Three places your code can live:

Working directory → Staging area → Local repo → Remote (GitHub)

(your edits) (git add) (git commit) (git push)

Nothing leaves your laptop until `git push`. `init`, `add`, `commit`, `branch`, `checkout`, `merge` — all 100% local.

2. Branching strategy — what orgs actually do

main is sacred. Nobody commits to it directly, ever. It should always be in a deployable/working state — in your case, always a state where `dbt run` succeeds cleanly.

Every unit of work gets its own branch. Not "one branch for the week" — one branch per logical change: a new model, a bug fix, a config change. This is the single most important habit. Small, focused branches → small, reviewable PRs → fast merges → less risk.

Rule of thumb: **if you can't describe the branch's purpose in five words, it's probably doing too much.**

Branch naming convention

<type>/<short-description>

Type	Use for
<code>feature/</code>	new functionality — new model, new metric, new integration

Type	Use for
<code>fix/</code>	bug fix
<code>chore/</code>	maintenance — dependency bumps, refactors, cleanup
<code>docs/</code>	documentation-only changes

Examples: `feature/add-revenue-metrics`, `fix/incorrect-join-fact-bookings`, `chore/upgrade-dbt-1-13`

3. The core commands — when and why

Command	What it does	When you use it
<code>git status</code>	Shows what's changed, staged, untracked	Constantly — before <i>every</i> add/commit
<code>git branch -a</code>	Lists all branches (local + remote)	Before switching, to see what exists
<code>git checkout -b <name></code>	Create + switch to a new branch	Starting new work
<code>git checkout <name></code>	Switch to an existing branch	Resuming work
<code>git add <file></code>	Stage a specific file	Preferred — deliberate, reviewable
<code>git add -A</code>	Stage everything changed	Fine for small, focused branches
<code>git commit -m "message"</code>	Snapshot staged changes, locally	After a logical unit of work is done
<code>git push -u origin <branch></code>	Push branch to GitHub, first time	Once, per new branch
<code>git push</code>	Push subsequent commits	Every time after the first push

Command	What it does	When you use it
<code>git pull</code>	Fetch + merge remote changes into current branch	Before starting new work on <code>main</code> , or when a branch is behind
<code>git log --oneline</code>	See commit history, condensed	Reviewing what's been done
<code>git diff</code>	See exact line-level changes, unstaged	Before <code>git add</code> , to review what you're about to stage

4. The full workflow, end to end

1. Start from an up-to-date main

```
git checkout main
```

```
git pull origin main
```

2. Create your branch for this specific piece of work

```
git checkout -b feature/add-revenue-metrics
```

3. Do the work — edit files, run dbt, iterate

```
dbt run --select fct_revenue
```

```
dbt test --select fct_revenue
```

4. Review what changed before staging

```
git status
```

```
git diff
```

5. Stage and commit — small, logical commits, not one giant dump

```
git add models/gold/fct_revenue.sql models/gold/schema.yml
```

```
git commit -m "Add fct_revenue model with margin calculation"
```

6. Push to GitHub

```
git push -u origin feature/add-revenue-metrics
```

7. Open a Pull Request on GitHub (compare feature/add-revenue-metrics -> main)

8. After review/approval, merge via GitHub UI (not CLI, typically)

9. Clean up locally

```
git checkout main
```

```
git pull origin main
```

```
git branch -d feature/add-revenue-metrics # delete local branch, now merged
```

5. Commit message conventions

Short, imperative, describes *what the commit does* — not what you did:

-  Add incremental logic to fct_bookings
-  Fix null handling in dim_host join
-  updated stuff
-  fixed it
-  wip

Many orgs adopt **Conventional Commits** — a light structured format:

feat: add revenue metric to semantic layer

fix: correct join condition in fct_bookings

chore: bump dbt-snowflake to 1.13.0

docs: update README with new setup steps

This becomes machine-parseable — some CI pipelines auto-generate changelogs from these prefixes. Worth adopting now, cheap habit, pays off later.

6. Pull Requests — why they exist, what good ones look like

A PR is not bureaucracy — it's the **review checkpoint** before code enters the shared, trusted branch. At an org, this is where:

- A teammate catches a bug or bad assumption before it hits production
- CI runs automated checks (we'll build this in Phase 7) — `dbt run`, `dbt test`, linting — and blocks merge if anything fails
- There's a permanent, searchable record of *why* a change was made (title + description), not just *what* changed

Good PR description answers three things:

1. What changed
2. Why
3. How you validated it (e.g. "ran `dbt test`, all 14 tests passing")

Branch protection rules (GitHub Settings → Branches) are how orgs *enforce* this — e.g. "no direct pushes to `main`," "require 1 approval," "require status checks to pass before merge."

Worth setting this up on your repo now even solo — good habit, and it's literally the governance control you'll recommend at Smartsheet.

7. Merge strategies — which to pick

Strategy	What it does	When to use
Merge commit	Keeps every commit, adds a merge commit tying branches together	Default; preserves full granular history
Squash and merge	Collapses all branch commits into one clean commit on <code>main</code>	Branch has messy/WIP commits (typos, "oops," experiments) — keeps <code>main</code> history readable
Rebase and merge	Replays your commits individually on top of <code>main</code> , no merge commit	Want fully linear history, branch commits are already clean

Most common default for feature branches with iterative WIP commits: Squash and merge. Keeps `main`'s history as one entry per feature, easy to scan, easy to revert if needed.

8. Keeping your branch in sync with `main`

If `main` moves forward while you're still working on your branch (a teammate merges something), you have two options to catch up:

Option A: merge main into your branch

```
git checkout feature/your-branch
```

```
git pull origin main
```

Option B: rebase your branch onto latest main (cleaner history, more advanced)

```
git checkout feature/your-branch
```

```
git fetch origin
```

```
git rebase origin/main
```

Merge is safer and simpler — recommended while you're building confidence. **Rebase** rewrites your branch's commit history, which is powerful but can cause pain if you've already pushed the branch and someone else has pulled it. Rule of thumb: **never rebase a branch others are actively working from.**

9. Handling merge conflicts

Happens when the same lines were changed on both `main` and your branch. Git will tell you exactly which files:

```
git pull origin main # or rebase — either can trigger this
```

```
# CONFLICT (content): Merge conflict in models/gold/fct_revenue.sql
```

Open the conflicted file — git marks the conflicting sections:

```
<<<<<<< HEAD
```

your version

=====

incoming version

>>>>>> main

Manually edit to keep the correct version (or a combination), remove the
<<<<<</=====/>>>>>> markers, then:

git add <resolved-file>

git commit # completes the merge

10. What NOT to commit — hygiene checklist

- `profiles.yml`, `.env`, any credentials file — always in `.gitignore`
 - `.venv/` / virtual environment folders
 - `target/`, `dbt_packages/`, `logs/` — dbt's generated/build artifacts, always gitignored
 - Scratch/test files (`try`, `Untitled`, `tryy.sql`) — clean up before pushing, or use an `analyses/scratch/` pattern that's itself gitignored
-

Quick-reference cheat sheet

git status # what's changed

git checkout -b feature/x # new branch

git checkout main # switch branch

git add <file> # stage specific file

git commit -m "message" # snapshot locally

git push -u origin feature/x # first push of new branch

git push # subsequent pushes

git pull origin main # sync with remote main

git log --oneline -10 # recent history

git diff # unstaged changes

git branch -d feature/x # delete local branch after merge